

CSC310 Data Structures and Algorithm Analysis (Dr. Ng)

Programming Assignment: Infix and Postfix Expressions

Write a program to convert and evaluate arithmetic expressions in infix form.

Compilers evaluate expressions by converting the infix expression into postfix form first so that it can evaluate from left to right.

Part I: Write a procedure to convert an infix expression into its postfix form.

In **Infix** expressions, operator is in between operands. Operator precedence and associativity determines the order of evaluation of the expression. Precedence and associativity can be overwritten by pair of parentheses.

In **Postfix** expressions, there is no parenthesis or operator precedence. All operators are of the same precedence. Postfix expression is simply evaluated from left to right.

In this assignment, an infix expression **<expression>** is defined as (::=) as:

<expression> ::= <operand> |
 <expression><operator><operand> |
 (<expression>)

<operand> ::= A | B | C |... | Z | 0 | 1 |...| 9

<operator> ::= + | - | * | /

where in above, ::= denotes “**is defined as**”, | denoted “**or**”

Each operand is one character long. Operators * and / have the same precedence while + and – have the same precedence. However, * and / have higher precedence than + and -. All are left associate operators. Order of precedence can be changed by enclosing the operands by a pair of parenthesis. Spaces between operand and operator are **optional**.

You should check whether the input string is a correct representation of an expression according to the above definition of <expression>, especially the balancing of pairs of parenthesis (and).

Some examples are:

Infix:

A+B
A+B-C
A*B-C
A+B*C
(A+B)*C
(A+B)/(C-D)
A+B/C-D
A*B+(C-D/E)
A*B+C/D-E
A*B+C/(D-E)
A*(B+C/(D-E))

Postfix:

AB+
AB+C-
AB*C-
ABC*+
AB+C*
AB+CD-/
ABC/+D-
AB*CDE/-+
AB*CD/+E-
AB*CDE-/++
ABCDE-/++

A pseudo code algorithm for converting infix to postfix:

Read the expression left to right, one operator/operand at a time:

If it is an operand P, output P to the postfix expression.

If it is an operator X, put/save it aside, compare with all operators Y already saved:

If the incoming operator X has higher precedence than saved operator Y, then Y stays, otherwise Y comes out first to the postfix expression. This repeats for every saved operator already there.

If it is a "(", cover the saved operators and create a new region.

If it is a ")", get rid of one region of saved operators in reverse order.

If it comes to the end of input, output the saved operators in reverse order.

Try to do it by hand to see if you can convert the above infix expression to its corresponding postfix form.

Part II: Write a procedure to convert evaluate a postfix expression and print the final answer in **BOTH base 10 and base 36**.

To evaluate a postfix expression, read from left to right. Find the next operator, and apply it to the closest previous two operators (assuming it is a binary operator). Repeat this process until the end of the postfix expression.

Suppose that the operands represent integer digits to the **base 36** (0..9 A..Z) where A is 10, B is 11, ..., and Z is 35. and / is the integer division operator. Write a procedure evaluate a postfix expression. For examples:

12+3* → 9 or 9 (base 36)
123*+ → 7 or 7 (base 36)

12*3+	→ 5 or 5 (base 36)
12-34*+	→ 11 or B (base 36)
123*-4+	→ -1 or -1 (base 36)
15431-/++	→ 7 or 7 (base 36)
25*431-/++	→ 12 or C (base 36)
AB*	→ 110 or 32 (base 36)

Part III: Write a procedure to build an expression tree from a postfix expression and print the built expression tree in preorder, inorder and postorder.

You should use C++ for this assignment. You cannot call any library functions except those that deal with I/O and string handling functions.

Your program should execute interactively on the machines in the department's lab. **You program should provide the user with two-level menu system:**

The first level menu provides 3 choice options:

0. Quit.
1. Input an infix expression
2. Input a postfix expression

The second level menu of choice option (1) provides 2 choices:

0. Return to previous menu.
1. Print postfix expression only.
2. Print postfix expression, and print the preorder, inorder and postorder of the expression tree, and the final expression value in base BOTH base 10 and base 36.

The second level menu of choice option (2) provides 2 choices:

0. Return to previous menu.
1. Print the final expression value ONLY in base BOTH base 10 and base 36.
2. Print the preorder, inorder and postorder of the expression tree and the the expression value in base 10 and base 36.

Please test your program thoroughly and make sure it is free of errors. **You need to check if the given input is correct with respect to the respective context.** Points will be deducted for each error that I discover for you.

Please also pay attention to program design and follow all the good programming practice. I reserve the right to deduct points based on these criteria.

Submit: Program design
Hierarchical diagrams/ Structure charts
Procedure specification
Program listing
Testing cases with Justification
Results of test cases

Part IV (optional):

Use part I postfix result to build expression trees. Print the expression trees in Preorder, Inorder and Postorder to ensure that the expression tree is correct.

Incorporate the exponentiation operator $^$, which has higher precedence than $*$ and $/$